Servicii injectate, cu Query și Command

Ștefan · 5 exerciții în `stefan-academy-vanilla-csharp` · modelul complet e pe **Books**, tu îl repeți pe Courses, Enrolments și Users

Ce schimbăm: azi fiecare view își face singur serviciile (`new(BookRepository.Instance)`) și depinde de clase concrete. Le despărțim în trei: **Repository** în spatele unei interfețe · **Query** = serviciul care doar citește · **Command** = serviciul care schimbă date. Toate se construiesc **o singură dată**, în `Program`, și ajung la view-uri prin constructor.
Reguli: corpurile metodelor se **mută** neschimbate din `XService` în `XQueryService` / `XCommandService`; nu repari nimic altceva în același commit · un exercițiu per commit · `dotnet build` fără erori · aplicația se comportă exact ca înainte.
Datele: la fiecare „Gata când” pornești de la `Data/` din git: `git checkout -- Data/`. Textul **îngroșat** e ce tastezi tu.

Schema

```
Program.Main              ← singurul loc cu .Instance și cu new XService(...)
  │
  ├─ BookRepository.Instance  ──(Singleton: o singură listă, un singur fișier)
  │      │ ca IBookRepository
  │      ├──► new BookQueryService(repo)    : IBookQueryService     citește
  │      └──► new BookCommandService(repo)  : IBookCommandService   scrie
  │
  └─ new ViewLogIn(q, c) ──► new ViewStudent(user, q, c)
                               știe DOAR interfețele, nu face new pe nimic

Query   = Get…, Find…   → întoarce date, nu schimbă nimic
Command = Create/Update/Delete → schimbă date (Add, Update, Remove în repo)
```

De ce: view-ul nu mai știe *cine* îi dă cărțile, doar *ce* poate cere (interfața). Poți schimba repository-ul (fișier → bază de date) fără să atingi view-ul, iar un view care doar afișează primește doar `IQueryService`, deci nu poate șterge nimic din greșeală. E exact structura din proiectele Spring/ASP.NET: `service/query` + `service/command`, legate prin constructor.

Modelul — Books

1. Books: repository și servicii în spatele interfețelor

Unde: `Books/Repositories/`, `Books/Services/`, `Views/ViewStudent.cs:16–29`, `Views/ViewLogIn.cs`, `Program.cs`

Structura de la final:

```
Books/
├── Repositories/
│   ├── IBookRepository.cs        NOU
│   └── BookRepository.cs         : Repository<Book>, IBookRepository
└── Services/
    ├── Query/
    │   ├── IBookQueryService.cs    NOU
    │   └── BookQueryService.cs     NOU
    └── Command/
        ├── IBookCommandService.cs  NOU
        └── BookCommandService.cs   NOU      (BookService.cs se șterge)
```

Interfața repository-ului — exact metodele pe care le folosesc serviciile:

```
namespace stefan_academy_vanilla_charp.Books.Repositories
{
    public interface IBookRepository
    {
        Book FindById(Guid id);
        Book FindByStudentAndName(Guid studentId, string bookName);
        List<Book> FindByStudentId(Guid studentId);
        void Add(Book book);
        void Update(Book book);
        void Remove(Book book);
    }
}

public class BookRepository : Repository<Book>, IBookRepository
```

De verificat: `FindById`, `Add`, `Update` și `Remove` nu sunt scrise în `BookRepository`. Cum de compilează totuși `: IBookRepository`? Răspunsul în `RASPUNSURI.txt`.

Query și Command — două interfețe, câte o clasă fiecare:

```
namespace stefan_academy_vanilla_charp.Books.Services.Query
{
    public interface IBookQueryService
    {
        Book GetBook(Guid studentId, string bookName);
        List<Book> GetBooksByStudentId(Guid studentId);
    }

    public class BookQueryService : IBookQueryService
    {
        private readonly IBookRepository repository;

        public BookQueryService(IBookRepository repository)
        {
            this.repository = repository;
        }
        // GetBook, GetBooksByStudentId: corpurile din BookService, neschimbate
    }
}

namespace stefan_academy_vanilla_charp.Books.Services.Command
{
    public interface IBookCommandService
    {
        BookCreateResponse CreateBook(BookCreateRequest request);
        BookUpdateResponse UpdateBook(Guid id, BookUpdateRequest request);
        void DeleteBook(Guid id);
    }

    public class BookCommandService : IBookCommandService
    {
        private readonly IBookRepository repository;

        public BookCommandService(IBookRepository repository)
        {
            this.repository = repository;
        }
        // CreateBook, UpdateBook, DeleteBook: corpurile din BookService, neschimbate
    }
}
```

Legarea prin constructor — de sus în jos:

```
// Program.cs
BookRepository bookRepository = BookRepository.Instance;
bookRepository.Subscribe(new NotificatorTest());
bookRepository.Subscribe(new NotificatorFisier(Path.Combine("..", "..", "..", "Data", "jurnal.txt")));

IBookQueryService bookQueryService = new BookQueryService(bookRepository);
IBookCommandService bookCommandService = new BookCommandService(bookRepository);
...
ViewLogIn logIn = new(bookQueryService, bookCommandService);

// ViewLogIn.cs: câmpuri + constructor, apoi
ViewStudent viewer = new(user, bookQueryService, bookCommandService);

// ViewStudent.cs
private readonly IBookQueryService bookQueryService;
private readonly IBookCommandService bookCommandService;

public ViewStudent(User user, IBookQueryService bookQueryService,
                   IBookCommandService bookCommandService)
{
    loggedUser = user;
    this.bookQueryService = bookQueryService;
    this.bookCommandService = bookCommandService;
    ...
}
// citiri (GetBook, GetBooksByStudentId)      → bookQueryService
// scrieri (CreateBook, UpdateBook, DeleteBook) → bookCommandService
```

Gata când: `dotnet build` fără erori, iar

```
grep -rn "BookService\b\|BookRepository" Views
```

nu întoarce nimic. Student Stefan Scarlat, CARTI → 2 → `Dune Noua` de două ori, apoi 1:

```
Tastati numele cartii: Dune Noua
NotificatorTest Book created successfully
Cartea Dune Noua adaugata cu succes la data de …
Tastati numele cartii: Dune Noua
Cartea "Dune Noua id student…" e deja in lista.
Nume carte: Cartea1 , …
Nume carte: Alchimistul , …
Nume carte: Alchimistul , …
Nume carte: Dune Noua , …
```

Modificarea și ștergerea cărții dau aceleași mesaje ca înainte.

La fel, pe celelalte trei

2. Courses

Unde: `Courses/Services/CourseService.cs`, `Courses/Repositories/CourseRepository.cs`, `Views/ViewStudent.cs:27`

Cerința:

1. `ICourseRepository` cu metodele pe care le folosește `CourseService` azi (`FindById`, `Courses`, `FindByName`, `GetCourseListByCourseId`, `CoursesByDepartment`, `Add`, `Update`, `Remove`).
2. `ICourseQueryService` + `CourseQueryService`: `GetCourse`, `GetCourses`, `GetByName`, `GetCourseListByCourseId`, `GetCoursesByDepartment`.
3. `ICourseCommandService` + `CourseCommandService`: `CreateCourse`, `UpdateCourse`, `DeleteCourse`.
4. Construite în `Program`, date prin `ViewLogIn` la `ViewStudent`, ca la Books. `CourseService.cs` se șterge.

Gata când: build fără erori; `grep -rn "CourseService\b\|CourseRepository" Views` nu întoarce nimic; CURSURI → 1 și STATISTICI → 2 afișează exact ce afișau înainte.

3. Enrolments

Unde: `Enrolments/Services/EnrolmentService.cs`, `Enrolments/Repositories/EnrolmentRepository.cs`, `Views/ViewStudent.cs:28`

Cerința:

1. `IEnrolmentRepository`, tot cu metodele pe care le folosește serviciul azi.
2. `IEnrolmentQueryService` + `EnrolmentQueryService`: toate metodele `Get…`.
3. `IEnrolmentCommandService` + `EnrolmentCommandService`: `CreateEnrolment`, `UpdateEnrolment`, `DeleteEnrolment`.
4. Regula de împărțire, pentru orice metodă la care stai pe gânduri: **schimbă ceva în repository (`Add` / `Update` / `Remove`)? → Command. Altfel → Query.**

Gata când: build fără erori; `grep -rn "EnrolmentService\b\|EnrolmentRepository" Views` nu întoarce nimic; CURSURI → 2 (înscriere), → 3 (dezabonare) și STATISTICI → 1 dau aceleași mesaje ca înainte.

4. Users — trei view-uri

Unde: `Users/Services/UserService.cs`, `Views/ViewLogIn.cs:12–13`, `Views/ViewAdmin.cs:11`

Cerința:

1. `IUserRepository`, `IUserQueryService` (`GetUser`, `GetUserByFirstAndLastName`), `IUserCommandService` (cele 3 `Create…`, cele 3 `Update…`, `DeleteUser`).
2. `ViewLogIn` nu mai face `new(UserRepository.Instance)`: pentru login primește `IUserQueryService` (la login doar **citește**). Tot el construiește `ViewStudent` și `ViewAdmin`, deci primește și serviciile pe care trebuie să li le dea mai departe.
3. `ViewAdmin` primește prin constructor doar ce folosește. Ce folosește: Query, Command sau amândouă?

Gata când: build fără erori; `grep -rn "UserService\b\|UserRepository" Views` nu întoarce nimic; login ca Stefan Scarlat → `Logat cu succes!`; login ca admin + adăugare student merg ca înainte.

Singleton-ul rămâne — dar într-un singur loc

5. `.Instance` doar în `Program`

Unde: cele 4 repository-uri și `Program.cs`

De ce mai avem nevoie de Singleton: fiecare repository citește fișierul în constructor și ține lista în memorie. Două instanțe de `BookRepository` = două liste: o carte adăugată prin una nu există în cealaltă, iar ultima care face `Save` o șterge pe a primei. Singleton-ul garantează **o singură listă**; injecția garantează că toată lumea primește **aceeași** listă.

Cerința:

1. Constructorul fiecărui repository rămâne `private`, iar `Instance` rămâne singura cale de a-l obține.
2. `.Instance` se scrie doar în `Program.cs`, câte o dată pentru fiecare repository. Serviciile și view-urile nu-l mai cheamă deloc.
3. `Program` e locul unde se vede tot desenul: 4 repository-uri → serviciile → `ViewLogIn`. Construiești doar serviciile pe care le folosește un view. Azi niciun view nu creează, nu modifică și nu șterge cursuri, deci `CourseCommandService` există, dar nu se construiește încă.

Gata când:

```
grep -rn "\.Instance" --include="*.cs" . | grep -v "static"
```

întoarce exact 4 rânduri, toate din `Program.cs`.

> În `RASPUNSURI.txt`: (1) Scrie `new BookRepository()` în `Program`. Ce spune compilatorul și de ce? (2) Dacă `BookQueryService` și `BookCommandService` ar primi fiecare *alt* `BookRepository`, ce ar afișa CARTI → 1 după o adăugare? (3) Ce view primește doar Query? De ce e bine că nu primește și Command?